EXP 1:

QUESTIONS 5 / 5 completed

Closest Pair Brute F

CLOSEST PAIR BRUTE FORCE

A city planner has N locations marked by (x,y) coordinates and wants to find which two locations are nearest to each other to connect with a direct road. Write a program that finds the closest pair of points among N 2D points using brute-force

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <math.h>
#include <float.h>

typedef struct {
    double x;
    double y;
} Point;

double calculateDistance(Point p1, Point p2) {
    return sqrt(pow(p1.x - p2.x, 2) + pow(p1.y - p2.y, 2));
}

int main() {
    int N;

    // Reading N
    if (scanf("%d", &N) != 1) return 0;

    Point points[N];
```

INPUT

```
4
0 0
3 4
1 1
6 8
```

EXP 2:

QUESTIONS 5 / 5 completed

Exhaustive Subset S

EXHAUSTIVE SUBSET SUM

A shopkeeper has N distinct coin denominations and wants to know if any combination of them adds up exactly to a customer's bill. Write a program that checks all possible subsets of an array to determine if any subset sums to a target value.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int main() {
    int n, target;

    printf("Enter N: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter the array elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter target: ");
    scanf("%d", &target);

    int totalSubsets = 1 << n; // 2^N
    int found = 0;

    for (int mask = 0; mask < totalSubsets; mask++) {
```

INPUT

```
4
2 3 7 8
11
```

EXP 3:

QUESTIONS 5 / 5 completed

Brute Force TSP

BRUTE FORCE TSP

A delivery driver must visit 4 cities exactly once and return to the start, minimizing total distance, and the fleet manager wants the guaranteed-optimal route checked by trying every possibility. Write a program that solves the Travelling Salesman Problem for

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <limits.h>

int N;
int dist[5][5];
int min_distance = INT_MAX;
int best_route[6];
int current_route[6];
int visited[5];

void findTSP(int count, int current_dist, int start_city) {
    // Base case: All cities visited
    if (count == N) {
        // Add distance to return to the starting city
        int final_dist = current_dist + dist[current_route[count - 1]][start_city];

        if (final_dist < min_distance) {
            min_distance = final_dist;
            // Store the best route including the return to start
            for (int i = 0; i < N; i++) {
```

INPUT

```
4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
```

EXP 4:

QUESTIONS 5 / 5 completed

Collinear Points Ch

COLLINEAR POINTS CHECK

A surveyor has marked N points on a map and needs to check, by brute force, whether any three of them lie exactly on a straight line (which would mean a redundant marker). Write a program that checks all triples of 2D points to find any 3 that are

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

struct Point {
    int x;
    int y;
};

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

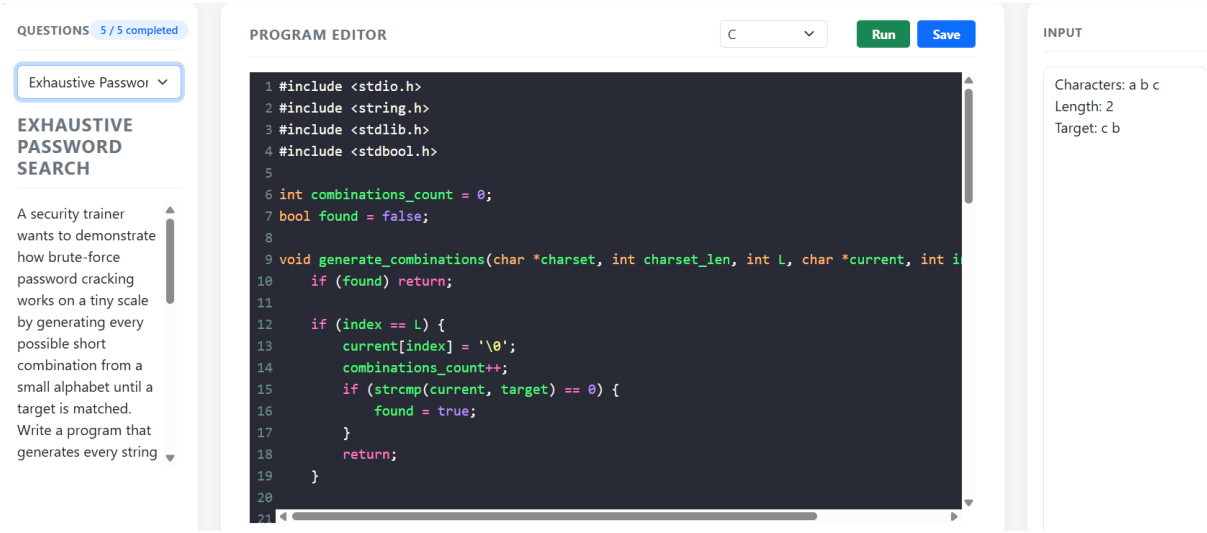
    struct Point points[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

    int found = 0;
    for (int i = 0; i < n - 2 && !found; i++) {
        for (int j = i + 1; j < n - 1 && !found; j++) {
            for (int k = j + 1; k < n && !found; k++) {
```

INPUT

```
4
1 1
2 2
3 3
5 1
```

EXP 5:



QUESTIONS 5 / 5 completed

Exhaustive Passwor

EXHAUSTIVE PASSWORD SEARCH

A security trainer wants to demonstrate how brute-force password cracking works on a tiny scale by generating every possible short combination from a small alphabet until a target is matched. Write a program that generates every string

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>

int combinations_count = 0;
bool found = false;

void generate_combinations(char *charset, int charset_len, int L, char *current, int i
    if (found) return;

    if (index == L) {
        current[index] = '\0';
        combinations_count++;
        if (strcmp(current, target) == 0) {
            found = true;
        }
        return;
    }
```

INPUT

Characters: a b c
Length: 2
Target: c b